

Object Oriented Programming

Câu hỏi

Class trong Java dùng để làm gì?

- A. Lưu dữ liệu tạm thời
- B. Là khuôn mẫu tạo Object
- C. Thực hiện vòng lặp
- D. Khai báo package

 Đáp án: B

Câu hỏi

Access Modifier nào chỉ cho phép truy cập bên trong chính class đó?

- A. public
- B. protected
- C. default
- D. private

 Đáp án: D

Câu hỏi

Modifier nào dùng để khai báo thành viên thuộc về Class thay vì Object?

- A. final
- B. static
- C. private
- D. abstract

 Đáp án: B

Câu hỏi

Method nào sau đây không trả về giá trị?

- A. `public int sum()`
- B. `public String getName()`
- C. `public void printInfo()`
- D. `public double avg()`

 Đáp án: C

Câu hỏi

Constructor có đặc điểm nào sau đây?

- A. Có kiểu trả về
- B. Tên khác tên class
- C. Tự động gọi khi tạo object
- D. Chỉ được khai báo một lần

 Đáp án: C

Câu hỏi

Đâu là ví dụ đúng của Method Overloading?

A.

- void show()
- void show(int x)

B.

- int show(int x)
- boolean show(int x)

C.

- void show(int x)
- void show(int y)

D.

- void show(String x)
- void show(String y)



Đáp án: A

Câu hỏi

Cách gọi Static Method đúng là gì?

- A. object.method()
- B. ClassName.method()
- C. static.method()
- D. new method()

 Đáp án: B

Câu hỏi

Method đầu tiên được JVM gọi khi chạy chương trình là gì?

- A. start()
- B. execute()
- C. run()
- D. main()

 Đáp án: D

Câu hỏi

Kết quả của đoạn code sau là gì?

```
class Demo {  
    private int age;  
  
    public Demo() {  
        System.out.println(age);  
    }  
}
```

- A. Compile Error
- B. null
- C. 0
- D. Không in gì



Đáp án: C

Câu hỏi

Kết quả của đoạn code sau là gì?

```
public class Test {  
    public static void main(String[] args) {  
        System.out.println(args.length);  
    }  
}
```

Chạy:

```
java Test A B C
```

A. 0

B. 1

C. 2

D. 3



Đáp án: D

Câu hỏi

Modifier nào thường dùng nhất cho thuộc tính để đảm bảo tính đóng gói (Encapsulation)?

- A. private
- B. public
- C. protected
- D. default

 Đáp án: A

Câu hỏi

Nếu một class không định nghĩa bất kỳ constructor nào thì Java sẽ tự động tạo:

- A. Constructor nhận 1 tham số
- B. Constructor nhận tất cả thuộc tính
- C. Default Constructor
- D. Static Constructor

 Đáp án: C

SỰ THỪA KẾ (INHERITANCE)

Thừa kế là gì?

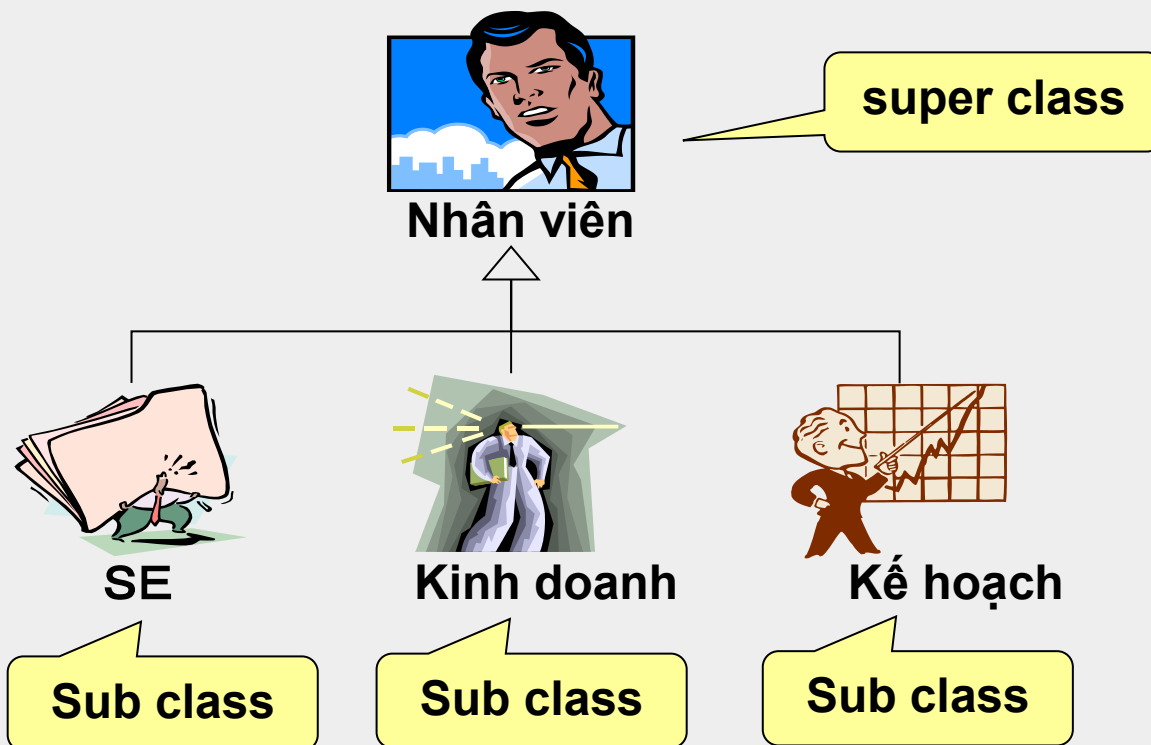
- Kế thừa là việc định nghĩa class mới đã kế thừa class sẵn có.
- Nếu sử dụng kế thừa thì có thể mở rộng chức năng của class sẵn có.
- Gọi class có những yếu tố chung thông thường là super class và gọi các class đã add thêm chức năng riêng biệt là sub class.
 - Cũng có trường hợp gọi super class là class cha, base class, etc...
 - Cũng có trường hợp gọi sub class là class con, class phái sinh, etc....

【Những điều có thể thực hiện ở subclass】

- Sub class có thể kế thừa tính chất (thuộc tính và thao tác) của super class.
- Sub class có thể add thêm tính chất mới (thuộc tính và thao tác) mà super class không có.
- Sub class cũng có thể định nghĩa lại (thay thành tính chất riêng, override) mà không kế thừa tính chất của super class.

Thừa kế là gì?

Có quan hệ generation, thừa kế thuộc tính và hoạt động của upper class (super class) sang lower class (sub class). Định nghĩa class mới mà có thừa kế class có sẵn.



Định nghĩa super class

■ Định nghĩa super class

● Cú pháp cơ bản

Định nghĩa super class hoàn toàn giống với phương pháp định nghĩa class đã xem ở chương 2.

```
Modifier class Class name {
```

```
    // Định nghĩa thuộc tính (Field)
```

```
    // Định nghĩa thao tác (Method)
```

```
}
```

Trong phần định nghĩa thuộc tính và hoạt động của super class, nếu cần thì sử dụng modifier protected.

● Ví dụ

```
public Employee class {  
    // Định nghĩa field và method
```

Protected modifier

- Một trong modifier có thể chỉ định thuộc tính và thao tác là protected.

```
public Employee class {  
    protected String empld;  
    protected String empName;  
    (Lược)  
}
```

- Nếu chỉ định là 「private String empld」 thì không thể access trực tiếp trong constructor của Sales class và class SE
- Nếu chỉ định là 「protected String empld」 thì có thể access trực tiếp trong constructor của Sales class và class SE

- Mặc dù muốn tránh việc công khai ra bên ngoài nhưng khi định nghĩa subclass, trường hợp có thuộc tính và thao tác của super class muốn access trực tiếp thì sẽ chỉ định protected modifier.

Định nghĩa sub class (1)

■ Định nghĩa sub class

● Point (「Những khả năng thực hiện ở sub class」được thể hiện bằng Java như thế nào)

• Class mới sẽ có thể thừa kế tính chất của class sẵn có (Thuộc tính và Thao tác).

→ Chỉ định 「**extends super class name**」 khi định nghĩa sub class

• Có thể add thêm thuộc tính mới (thuộc tính và thao tác)

→ Mô tả 「**Thuộc tính độc lập (Thuộc tính và thao tác)**」 vào trong định nghĩa class của sub class.

• Cũng có thể định nghĩa lại (chuyển sang tính chất riêng biệt) mà không kế thừa thuộc tính của class sẵn có.

→ **Override method**

Định nghĩa sub class(2)

■ Định nghĩa sub class(Thừa kế tính chất của super class)

● Cú pháp cơ bản

```
Modifier class Sub class name extends super class name {  
    // Định nghĩa tính chất riêng muốn add thêm
```

Chỉ bằng việc chỉ định 「extends super class name」 có thể thừa kế được tính chất (Biến và method) được định nghĩa trong super class.

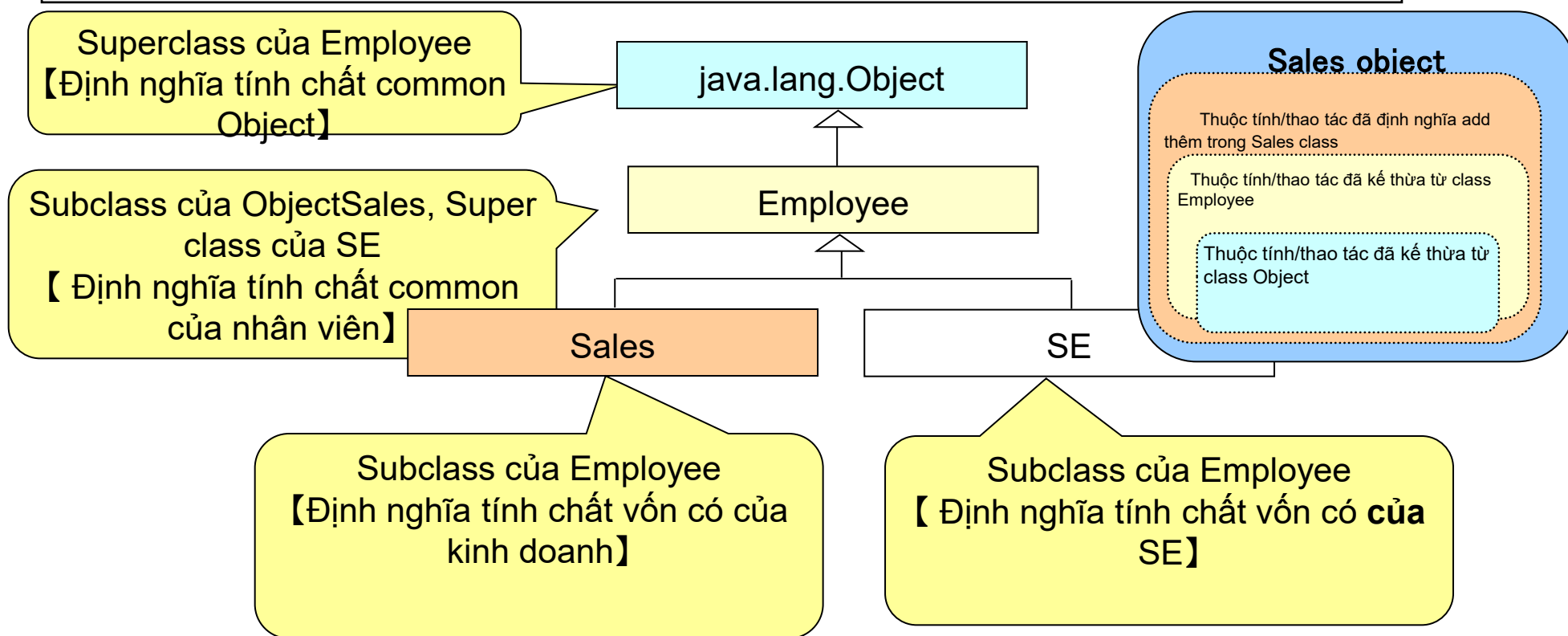
● Ví dụ

```
public Sales class extends Employee {  
    // Mô tả thuộc tính và thao tác add thêm  
}
```

Định nghĩa sub class(2)

```
public Employee class { // Trường hợp không có chỉ định của super class
    // thì coi là「extends java.lang.Object」
    // Định nghĩa tính chất (thuộc tính, thao tác) chung
```

```
public Sales class extends Employee {
    // Mô tả thuộc tính và thao tác add thêm vào Sales class vốn có }
```



Định nghĩa sub class(3)

■ Định nghĩa sub class (Add thêm thuộc tính, thao tác riêng)

● Ví dụ

Chỉ cần định nghĩa tính chất riêng (Thuộc tính và thao tác) muốn add thêm vào sub class là được.

```
public Sales class extends Employee {
```

```
    private int customerCount;
```

Thuộc tính riêng
của Sales

```
    public Sales(String id, String ename, int cnt) {  
        // Set giá trị khởi tạo (Lược)  
    }
```

```
    public void addCount() {  
        customerCount++;  
    }
```

Thao tác riêng
của Sales

```
    public int getCount() {  
        return customerCount;  
    }
```

```
}
```

Override method

■ Override

● Override là gì?

Định nghĩa lại một cách riêng biệt tại phía sub class về chức năng có trong super class.

Về chức năng tên mà super class đang có thì thay vì thừa kế nguyên, có thể thay đổi nội dung hoạt động bên phía sub class sao cho sub class hoạt động độc lập!

● Biểu hiện override trong Java...

Định nghĩa ở sub class các method của super class, method mà giá trị trả về và signature (Tên và List các đối số) là hoàn toàn giống nhau

Đối số giống nhau có nghĩa là 「Số lượng của đối số」 「Data type」 「Trình tự định nghĩa đối số」 hoàn toàn giống nhau.

Override và overload

Tiêu chí	Overloading	Overriding
Xảy ra ở đâu?	Cùng một lớp	Giữa lớp cha và lớp con
Tên phương thức	Giống nhau	Giống nhau
Tham số	Khác nhau	Giống nhau
Trả về	Có thể khác nhau	Phải giống hoặc kiểu con
Quyết định gọi lúc	Biên dịch (compile time)	Chạy (runtime)

Override

```

1. public Employee class {
2.     protected String empld;      // Mã số nhân viên
3.     protected String empName;    // Họ tên
4.         (Lược)
5.     public void calcSalary() {
6.         System.out.println("Employee calcSalary");
7.     }
8. }
```

```

1. public Sales class extends Employee {
2.     private int customerCount;    // số lượng customer
3.
4.     public Sales(String id, String name, int cnt) {
5.         empld = id;
6.         empName = name;
7.         customerCount = cnt;
8.     }
9.         (Lược)
10. }
```

**kế thừa
calcSalary của
Employee**

```

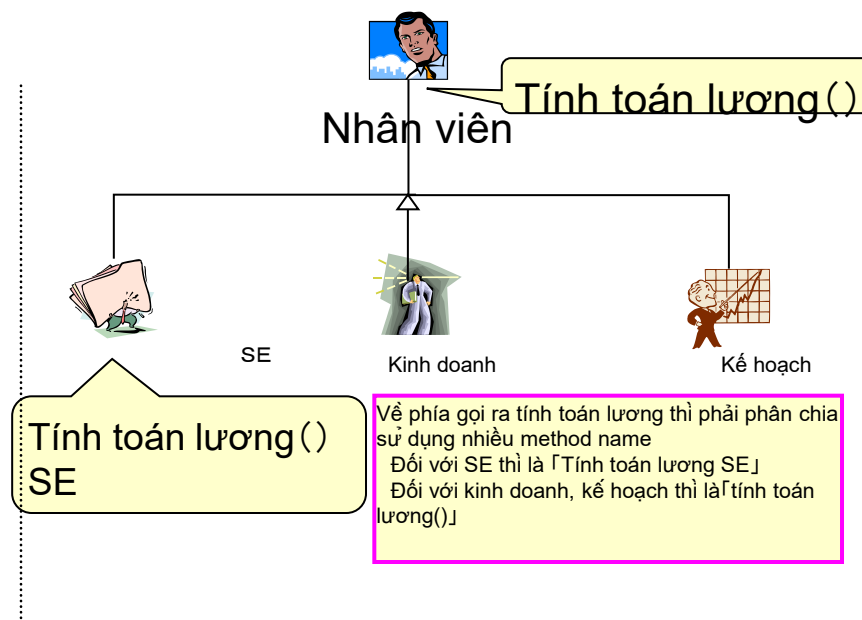
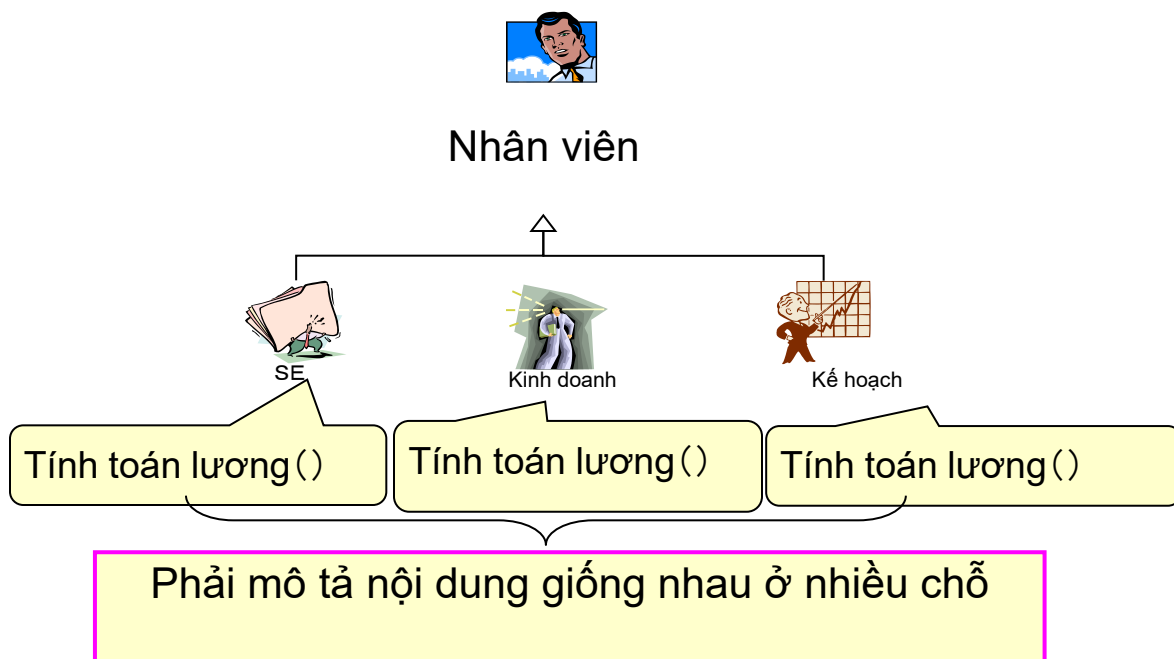
1. public class SE extends Employee {
2.     private String specializedField; // Lĩnh vực chuyên môn
3.
4.     public SE(String id, String ename, String f) {
5.         empld = id;
6.         empName = ename;
7.         specializedField = f;
8.     }
9.         (Lược)
10.     public void calcSalary() {
11.         System.out.println("SE calcSalary");
12.     }
13. }
```

**Định nghĩa lại một cách độc lập chứ
không kế thừa calcSalary của Employee**

Override

- Định nghĩa những phần có nội dung tương tự trong nhiều subclass
- Phía sử dụng method sẽ phân chia sử dụng nhiều method name

- Xử lý common trong nhiều subclass sẽ định nghĩa trong superclass (mô tả chỉ ở 1 chỗ)
- Phía sử dụng method có thể sử dụng common method name (method name giống với toàn bộ subclass)



@Override annotation

- Annotation
- Là việc thực hiện gán ý nghĩa bổ sung đối với program element như class, method, field, etc... Comment là có thể truyền đạt thông tin bổ sung khác cho compile (tool).
- Ví dụ, trường hợp override method của superclass trong subclass, nếu chỉ định @Override annotation trong method của subclass mà không định nghĩa method được override trong superclass thì sẽ bị compile error và có thể check khi compile.
- Annotation là function từ J2SE5.0.

this và super

■ this và super

● this

Chỉ bản thân object hiện tại đang thực hiện xử lý (Instance)

```
public Employee(String empld, String empName) {  
    this.empld = empld;  
    this.empName = empName;  
}
```

Gán giá trị đã nhận bằng đối số (empld) vào biến instance (this.empld) của class Employee.

● super

Chỉ đến super class của class mà hiện tại đang thực hiện xử lý

```
public void printInfo() {  
    super.printInfo();  
    System.out.println("customerCount:" + customerCount);  
}
```

Gọi ra printInfo method của super class (Employee) trong printInfo method của sub class (Sales).

Gọi ra constructor của chính class đó

■ Gọi ra constructor của chính class đó

● Cú pháp cơ bản

this(List các đối số);

Trong trường hợp chỉ định thì nhất định là phải mô tả làm xử lý phần đầu của constructor.

● Ví dụ .

```
public Employee class {  
    private String empld;  
    private String empName;
```

```
    public Employee() {  
        this("123456", "Yamada ichiro");  
    }
```

```
    public Employee(String id, String ename) {  
        empld = id;  
        empName = ename;  
    }
```

【Tham khảo】 Mô tả cho tới đây
empld = "123456";
empName = "Yamada ichiro";

Gọi ra constructor của super class

■ Gọi ra constructor của super class

● Cú pháp cơ bản

super(List các đối số);

Trường hợp chỉ định thì nhất định phải mô tả làm xử lí đầu của constructor.

● Ví dụ

【Constructor của super class】

```
public Employee(String id, String ename) {  
    empId = id;  
    empName = ename;  
}
```

【Constructor của sub class】

```
public Sales(String id, String ename, int cnt) {  
    super(id, ename);  
    customerCount = cnt;  
}
```

【Tham khảo】 Mô tả cho đến đây
empId = id;
empName = ename;